

Constrained Optimization-Based Neuro-Adaptive Control (CONAC) for Euler-Lagrange Systems Under Weight and Input Constraints

Myeongseok Ryu, Donghwa Hong, and Kyunghwan Choi

Abstract—This study presents a constrained optimization-based neuro-adaptive control (CONAC) for unknown Euler-Lagrange systems subject to weight and convex input constraints. A deep neural network (DNN) is employed to approximate the ideal stabilizing control law while simultaneously learning the unknown system dynamics and addressing both types of constraints within a unified constrained optimization framework. The adaptation law of DNN weights is formulated to solve the defined constrained optimization problem, ensuring satisfaction of first-order optimality conditions at steady state. The controller's stability is rigorously analyzed using Lyapunov theory, guaranteeing bounded tracking errors and bounded DNN weights. The proposed controller is validated through a real-time implementation on a 2-DOF robotic manipulator, demonstrating its effectiveness in achieving accurate trajectory tracking while satisfying all imposed constraints.

Index Terms—Neuro-adaptive control, constrained optimization, deep neural network, input constraint, weight constraint.

NOTATION

The Kronecker product is denoted by $\otimes$ [1, Chap. 7, Def. 7.1.2]. A vector and a matrix are denoted by $\mathbf{x} = [x_i]_{i \in \{1, \dots, n\}} \in \mathbb{R}^n$ and $\mathbf{A} := [a_{ij}]_{i \in \{1, \dots, n\}, j \in \{1, \dots, m\}} \in \mathbb{R}^{n \times m}$, respectively. Given a matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\text{row}_i(\mathbf{A})$ denotes the i th row of $\mathbf{A}$. For the matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\text{vec}(\mathbf{A}) := (\text{row}_1(\mathbf{A}^\top), \dots, \text{row}_m(\mathbf{A}^\top))^\top \in \mathbb{R}^{nm}$ denotes the vectorization of $\mathbf{A}$. $\lambda_{\min}(\mathbf{A})$ denotes the minimum eigenvalue of the matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. The $n \times n$ identity and $n \times m$ zero matrices are denoted by $\mathbf{I}_n$ and $\mathbf{0}_{n \times m}$, respectively.

I. INTRODUCTION

A. Background

MANY engineering systems, including those in aerospace, robotics, and automotive applications, can be modeled using Euler-Lagrange systems. These systems are governed by dynamic equations derived from energy principles and describe the motion of mechanical systems with constraints. In practice, however, such systems often

exhibit uncertainties due to unmodeled dynamics, parameter variations, or external disturbances. These uncertainties can significantly degrade control performance and, in some cases, lead to instability. In worst cases, any prior knowledge of the system dynamics may not be available, making it challenging to design effective controllers. To address these challenges, adaptive control methods have been widely employed to ensure robust performance for uncertain or unknown systems [2], [3].

More recently, neuro-adaptive control (NAC) approaches have been introduced to approximate uncertain or unknown system dynamics or entire control laws using neural networks (NNs) [4], [5]. NNs are well-known for their universal approximation property, which allows them to approximate any smooth function over a compact set with minimal error. Various types of NNs have been utilized in NAC, including simpler architectures like single-hidden layer (SHL) NNs [6]–[8] and radial basis function (RBF) NNs [9], [10], as well as more complex models like deep NNs (DNNs) [11] and their variations. Conventionally, SHL and RBF NNs are often employed to approximate uncertain or unknown system dynamics or controllers due to their simplicity [8], [12]–[14]. However, since DNNs offer greater expressive power, making them more effective for complex system approximations [15], variations of DNNs, such as long short-term memory (LSTM) networks for time-varying dynamics [16], physics-informed NNs (PINNs) for leveraging physical system knowledge [17], and convolutional NNs (CNNs) for learning historical sensor data [18], have further extended the capabilities of NAC systems.

A critical aspect of NAC is the NN weight adaptation law, which governs how NN weights are updated. Most studies derived these laws using Lyapunov-based methods, ensuring the boundedness of the tracking error and weight estimation error, thus maintaining system stability under uncertainty.

However, two significant challenges persist in using NNs for adaptive control. First, the boundedness of NN weights is not inherently guaranteed, which can result in unbounded outputs. When NN outputs are used directly in the control law, this may lead to excessive control inputs, violating input constraints. Such constraints are commonly encountered in industrial systems, where actuators are limited by physical and safety requirements in terms of amplitude, rate, or energy [19]. Failing to address these constraints can degrade control performance or even destabilize the system.

Therefore, addressing these two key issues—ensuring weight boundedness and satisfying input constraints—is essential for the reliable design of NAC. The following section provides a detailed review of existing solutions to these

Myeongseok Ryu and Kyunghwan Choi are with the Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology (KAIST), Daejeon 34051, Korea (e-mail: dding_98@kaist.ac.kr and kh.choi@kaist.ac.kr).

Donghwa Hong is with the Department of Mechanical and Robotics Engineering, Gwangju Institute of Science and Technology (GIST), Gwangju 61005, Korea (e-mail: fairytaletf00@gm.gist.ac.kr).

This work was supported in part by a Korea Research Institute for Defense Technology planning and advancement (KRIT) grant funded by Korea government DAPA (Defense Acquisition Program Administration) (No. KRIT-CT-22-087, Synthetic Battlefield Environment based Integrated Combat Training Platform) and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025- 00554087). (Corresponding author: Kyunghwan Choi)

challenges.

B. Literature Review

1) *Ensuring Weight Norm Boundedness*: A common challenge in NAC is maintaining the boundedness of the NN weights to ensure stability. In many studies, projection operators were employed to enforce upper bounds on the weight norms, ensuring that the weights do not grow unboundedly. For example, in [11], [16], [20], projection operators were used to constrain the weight norms to remain below predetermined constants. However, these constants were often selected as large as possible due to the lack of theoretical guarantees regarding the global optimality of the weight values. While this approach ensured that the NN remained stable, it did not necessarily result in optimal performance.

In addition to projection operators, some studies utilized modification techniques like σ -modification [10] and ϵ -modification [13], [14]. These methods ensured that the NN weights remained within an invariant set by incorporating stabilizing functions into the adaptation law. Although these techniques were effective in ensuring boundedness and avoiding weight divergence, they similarly lacked a formal analysis of the optimality of the adapted weights by biasing the weights toward the origin [21]. This leaves room for improvement in terms of performance optimization.

2) *Satisfying Input Constraints*: The second major issue is satisfying input constraints, particularly in systems where actuators are subject to physical limitations. This issue becomes more pronounced in NAC architectures where NNs are used to augment conventional model-based controllers—such as feedback linearization or backstepping—rather than acting as the sole control input. In such cases, if the system model is inaccurate or the dynamics targeted by the NN are already contributing to stability, the combined control effort may result in overly aggressive inputs, increasing the risk of input saturation [22].

To address the aforementioned issue, several approaches have been proposed in [12]–[14], [23]–[26] to handle input constraints in NAC systems. Particularly, the auxiliary systems have been widely introduced. In [23]–[25], the auxiliary states were generated whenever input saturation was detected, and these states were used as feedback terms in the control law to directly compensate for the effects of input saturation constraints. On the other hands, in [12], [13], [26], the auxiliary systems were incorporated into the adaptation law by adding the auxiliary states to the feedback error for learning. This approach helped the NN reduce input saturation by indirectly regulating the auxiliary states, during the adaptation process.

However, these approaches typically handle input bound constraints on a per-input basis, i.e., applying bounds to each scalar control variable individually, and may not account for more complex, nonlinear constraints, like input norm constraints, which are commonly found in physical systems such as robotic actuators or motor systems.

Furthermore, from a learning perspective, augmenting conventional control methods with NNs by simply adding NN

outputs to existing control inputs can interfere with the adaptation process. This occurs because the adaptation law typically depends on the feedback error, which becomes insensitive to the NN output when the conventional controller dominates the system behavior. As a result, the NN ends up counteracting the conventional control effort rather than directly learning the system dynamics, thereby weakening its ability to adapt. To address this, input constraints should be handled within a unified adaptive framework—such as those proposed in [12], [13], [26]—where both adaptation and constraint satisfaction are treated jointly without relying on conventional control inputs.

3) *Potential of Constrained Optimization*: A promising approach to overcoming the limitations of the existing methods lies in constrained optimization. By formulating the NAC problem as an unified optimization problem with constraints, it is possible to adapt the NN weights while minimizing an objective function, e.g., tracking error, subject to both weight and input constraints, simultaneously. Constrained optimization provides a theoretical framework for defining optimality and presents numerical methods for finding solutions that satisfy the constraints [27].

In existing literature, constrained optimization techniques, such as the Augmented Lagrangian Method (ALM) [28] and the Alternating Direction Method of Multipliers (ADMM) [29], [30], have been used to train NNs offline. However, to the best of the authors' knowledge, no prior work has applied constrained optimization to NAC systems with real-time weight adaptation. This gap suggests that constrained optimization could be key to addressing both weight boundedness and input constraints in a unified, theoretically grounded framework, particularly in real-time NAC.

C. Contributions

The main contributions of this study are listed as follows:

- A constrained optimization-based neuro-adaptive control (CONAC) is proposed, where the control problem is formulated as a constrained optimization problem. In this formulation, weight and convex input constraints are incorporated as inequality constraints, while minimizing a given objective function.
- Since the proposed CONAC approximates the entire ideal control law without any conventional controller, prior knowledge of the system dynamics or the system identification process for nominal conventional controller design can be avoided, which is often difficult and time-consuming to implement in practice.
- The adaptation laws for DNN weights and Lagrange multipliers are systematically derived to solve the defined problem, using constrained optimization theory. These laws guarantee convergence to the first-order optimality conditions at steady state, specifically the Karush–Kuhn–Tucker (KKT) conditions, as described in [27, Chap. 12, Thm. 12.1].
- The proposed CONAC's stability is rigorously analyzed using Lyapunov theory, ensuring that the tracking error

and DNN weights remain bounded. This analysis guarantees that the proposed CONAC can achieve stable online adaptation.

- The proposed CONAC is implemented in real-time on a 2-DOF robotic manipulator, demonstrating its effectiveness in achieving accurate trajectory tracking while satisfying all imposed constraints. The experimental results validate the theoretical findings and confirm the practical applicability of the proposed CONAC.

This study extends our preliminary work in [21], where an inequality constraint on the weight norm was introduced within a constrained optimization framework to ensure boundedness of the NN weights. In addition to the weight constraints considered in [21], the present work also incorporates input constraints into the same framework to address the convex input saturation problem. Moreover, while the previous study was limited to numerical simulations, the proposed CONAC is experimentally validated through real-time implementation on a 2-DOF robotic manipulator.

D. Organization

The remainder of this paper is organized as follows. Section II presents the target system and control objective. Section III introduces the proposed CONAC and the architecture of DNN used in the controller. In Section IV, the adaptation law is developed. Section V analyzes a stability of CONAC. Section VI presents a real-time implementation of CONAC on a 2-DOF robotic manipulator. Finally, Section VII concludes the paper and discusses potential future work. Candidates for convex input constraints are provided in Appendix A.

II. PROBLEM FORMULATION

A. Model Dynamics and Control Objective

We consider an unknown Euler-Lagrange system modeled as

$$\mathbf{M} \frac{d^2}{dt^2} \mathbf{q} + \mathbf{V}_m \frac{d}{dt} \mathbf{q} + \mathbf{F} + \mathbf{G} + \boldsymbol{\tau}_d = \text{sat}(\boldsymbol{\tau}), \quad (1)$$

where $\mathbf{q} \in \mathbb{R}^n$ denotes the generalized coordinate, $\boldsymbol{\tau}_d$ represents the disturbance and $\boldsymbol{\tau} \in \mathbb{R}^n$ denotes the generalized control input. The terms $\mathbf{M} := \mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$, $\mathbf{V}_m := \mathbf{V}_m(\mathbf{q}, \frac{d}{dt} \mathbf{q}) \in \mathbb{R}^{n \times n}$, $\mathbf{F} := \mathbf{F}(\frac{d}{dt} \mathbf{q}) \in \mathbb{R}^n$, and $\mathbf{G} := \mathbf{G}(\mathbf{q}) \in \mathbb{R}^n$ represent the unknown inertia matrix, Coriolis/centripetal matrix, friction vector, and gravity vector, respectively. The function $\text{sat}(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ represents the inherent physical limitations of the actuators such that $\|\text{sat}(\boldsymbol{\tau})\| \leq \bar{\tau}$, where $\bar{\tau} \in \mathbb{R}_{>0}$ denotes the maximum norm of control input.

In this study, the saturation function $\text{sat}(\cdot)$ is assumed to be convex, which is practically common in many engineering systems. This is because actuator limits are typically convex—for example, minimum and maximum torque constraints—which define box- or ball-shaped bounds in the input space. To account for these limitations, it is essential to incorporate physically motivated constraints into the controller design. Appendix A introduces candidate constraints that can be applied to ensure compliance with these physical limitations.

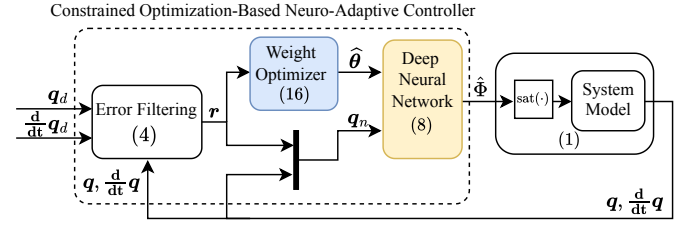


Fig. 1: Architecture of the proposed constrained optimization-based neuro-adaptive controller (CONAC).

The Euler-Lagrange system (1) satisfies several important physical properties, as presented in [4, Chap. 3, Tab. 3.2.1]. The key properties are introduced below:

Property 1. The inertia matrix $\mathbf{M}$ is symmetric, positive definite, and bounded.

Property 2. The Coriolis/centripetal matrix $\mathbf{V}_m$ can always be selected, so that the matrix $\frac{d}{dt} \mathbf{M} - 2\mathbf{V}_m$ is skew-symmetric, i.e., $\mathbf{x}^\top (\frac{d}{dt} \mathbf{M} - 2\mathbf{V}_m) \mathbf{x} = 0$, $\forall \mathbf{x} \in \mathbb{R}^n$.

Property 3. The disturbance $\boldsymbol{\tau}_d$ is bounded, i.e., $\|\boldsymbol{\tau}_d\| \leq \bar{\tau}_d$, where $\bar{\tau}_d \in \mathbb{R}_{\geq 0}$.

The control objective is to develop a NAC that enables $\mathbf{q}$ to track a continuously differentiable desired trajectory $\mathbf{q}_d := \mathbf{q}_d(t) : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$, compensating for the unknown system dynamics while addressing the imposed constraints. The desired trajectory $\mathbf{q}_d(t)$ is supposed to satisfy the following assumption:

Assumption 1. The desired trajectory $\mathbf{q}_d(t)$ is assumed to be bounded, i.e., $\|\mathbf{q}_d(t)\| \leq \bar{q}_d \in \mathbb{R}_{>0}$, and available for designing a bounded control input.

III. CONTROL LAW DEVELOPMENT

The architecture of the proposed CONAC consists of a DNN that functions as a NAC, and a weight optimizer for the DNN, as illustrated in Fig. 1. Before proceeding to the controller design, some mathematical preliminaries are revisited in Section III-A. Section III-B introduces the NAC, followed by the DNN model in Section III-C. The constrained optimization-based weight adaptation law is presented in Section IV.

A. Mathematical Preliminaries

The following proposition and lemma will be used in subsequent sections:

Proposition 1 (see [1, Chap. 7, Prop. 7.1.9]). For a matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ and a vector $\mathbf{b} \in \mathbb{R}^n$, the following property holds:

$$\mathbf{A}^\top \mathbf{b} = \text{vec}(\mathbf{A}^\top \mathbf{b}) = \text{vec}(\mathbf{b}^\top \mathbf{A}) = (\mathbf{I}_m \otimes \mathbf{b}^\top) \text{vec}(\mathbf{A}). \quad (2)$$

Lemma 1. For a Lyapunov function $V := V(\mathbf{x}) \geq 0$ with $\mathbf{x} \in \mathbb{R}^n$, if the time derivative of V is given by $\frac{d}{dt} V \leq -a_1 \mathbf{x}^\top \mathbf{x} + a_2 \mathbf{x}$ for some positive constants $a_1 \in \mathbb{R}_{>0}$ and $a_2 \in \mathbb{R}_{>0}$, then $\mathbf{x}$ is bounded as $\mathbf{x} \in \{\mathbf{x} \mid \|\mathbf{x}\| \leq \frac{a_2}{a_1}\}$.

Proof. The time derivative of V can be rewritten as

$$\frac{d}{dt}V \leq -a_1\|\mathbf{x}\|^2 + a_2\|\mathbf{x}\| = -a_1\left(\|\mathbf{x}\| - \frac{a_2}{2a_1}\right)^2 + \frac{a_2^2}{4a_1}. \quad (3)$$

According to (3), it can be concluded that $\frac{d}{dt}V$ is negative definite if $\|\mathbf{x}\| > \frac{a_2}{a_1}$ holds. In other words, $\mathbf{x}$ remains within the bounded set $\{\mathbf{x} \mid \|\mathbf{x}\| \leq \frac{a_2}{a_1}\}$, since $\frac{d}{dt}V$ is negative definite when $\mathbf{x}$ is outside the set. $\square$

B. Neuro-Adaptive Controller Design

Given that the Euler-Lagrange system (1) exhibits second-order dynamics, a filtered error $\mathbf{r} \in \mathbb{R}^n$ is introduced to convert the system into a first-order system, as follows:

$$\mathbf{r} := \frac{d}{dt}\mathbf{e} + \mathbf{\Lambda}\mathbf{e}, \quad (4)$$

where $\mathbf{e} := \mathbf{q} - \mathbf{q}_d$ denotes the tracking error, $\frac{d}{dt}\mathbf{e} := \frac{d}{dt}\mathbf{q} - \frac{d}{dt}\mathbf{q}_d$ represents the time derivative of the tracking error, and $\mathbf{\Lambda} \in \mathbb{R}_{>0}^{n \times n}$ is a user-designed filtering matrix. Since (4) is stable system, it implies that $\mathbf{e}$ is bounded if $\mathbf{r}$ is bounded.

Using $\mathbf{r}$, the system dynamics (1) can be rewritten as

$$\mathbf{M} \frac{d}{dt}\mathbf{r} = -\mathbf{V}_m\mathbf{r} - \mathbf{K}\mathbf{r} + \mathbf{f} - \boldsymbol{\tau}_d + \text{sat}(\boldsymbol{\tau}), \quad (5)$$

where $\mathbf{K} = \mathbf{K}^\top \in \mathbb{R}_{>0}^{n \times n}$ denotes an arbitrary unknown matrix and $\mathbf{f} := \mathbf{f}\left(\mathbf{q}, \frac{d}{dt}\mathbf{q}, \mathbf{q}_d, \frac{d}{dt}\mathbf{q}_d, \frac{d^2}{dt^2}\mathbf{q}_d\right) = \mathbf{K}\mathbf{r} + \mathbf{M}\left(-\frac{d^2}{dt^2}\mathbf{q}_d + \mathbf{\Lambda} \frac{d}{dt}\mathbf{e}\right) + \mathbf{V}_m\left(-\frac{d}{dt}\mathbf{q}_d + \mathbf{\Lambda}\mathbf{e}\right) - \mathbf{F} - \mathbf{G} \in \mathbb{R}^n$ denotes the lumped unknown system.

Consider the Lyapunov function $V_1 := \frac{1}{2}\mathbf{r}^\top \mathbf{M}\mathbf{r}$. Invoking Property 2, the time derivative of V_1 is

$$\begin{aligned} \frac{d}{dt}V_1 &= \mathbf{r}^\top \mathbf{M} \frac{d}{dt}\mathbf{r} + \frac{1}{2}\mathbf{r}^\top \frac{d}{dt}\mathbf{M}\mathbf{r} \\ &= \mathbf{r}^\top (-\mathbf{V}_m\mathbf{r} - \mathbf{K}\mathbf{r} + \mathbf{f} - \boldsymbol{\tau}_d + \text{sat}(\boldsymbol{\tau})) \\ &\quad + \frac{1}{2}\mathbf{r}^\top \frac{d}{dt}\mathbf{M}\mathbf{r} \\ &= -\mathbf{r}^\top \mathbf{K}\mathbf{r} + \mathbf{r}^\top (\mathbf{f} - \boldsymbol{\tau}_d + \text{sat}(\boldsymbol{\tau})) \\ &\quad + \frac{1}{2}\mathbf{r}^\top \left(\frac{d}{dt}\mathbf{M} - 2\mathbf{V}_m\right)\mathbf{r} \\ &\leq -\lambda_{\min}(\mathbf{K})\|\mathbf{r}\|^2 + \bar{\tau}_d\|\mathbf{r}\| + \mathbf{r}^\top (\mathbf{f} + \text{sat}(\boldsymbol{\tau})). \end{aligned} \quad (6)$$

Therefore, invoking Lemma 1, it can be concluded that the filtered error $\mathbf{r}$ is uniformly ultimately bounded with $\lim_{t \rightarrow \infty} \|\mathbf{r}\| \leq \frac{\bar{\tau}_d}{\lambda_{\min}(\mathbf{K})}$, provided that the lumped unknown system $\mathbf{f}$ can be perfectly cancelled by the control input $\boldsymbol{\tau}$, neglecting the effect of input saturation. Hence, the ideal control input $\boldsymbol{\tau}^*$ can be designed as $-\mathbf{f}$. However, $\boldsymbol{\tau}^*$ is not available in practice, since the system dynamics encapsulated in $\mathbf{f}$ are unknown.

To overcome this issue, a DNN is employed to approximate $\boldsymbol{\tau}^*$. Let $\boldsymbol{\Phi} := \boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta}) : \mathbb{R}^{l_0+1} \times \mathbb{R}^\Xi \rightarrow \mathbb{R}^n$ represent the DNN, where $\mathbf{q}_n \in \mathbb{R}^{l_0+1}$ is the DNN input vector, and $\boldsymbol{\theta} \in \mathbb{R}^\Xi$ is the vector of trainable weights. The architecture of DNN $\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta})$ will be presented in Section III-C, later. According to the universal approximation theorem for DNNs [31], $\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta})$ can approximate a nonlinear function, denoted as $\mathbf{g}(\cdot)$, with an ideal weight vector $\boldsymbol{\theta}^* \in \mathbb{R}^\Xi$ over a compact subset $\Omega_n \in \mathbb{R}^{l_0+1}$ to within ϵ -accuracy, such that $\sup_{\mathbf{q}_n \in \Omega_n} \|\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta}^*) - \mathbf{g}(\cdot)\| = \epsilon < \infty$. In this study, the ideal weight vector $\boldsymbol{\theta}^*$ is assumed to be bounded by Assumption 1 and [6, Assum. 1],

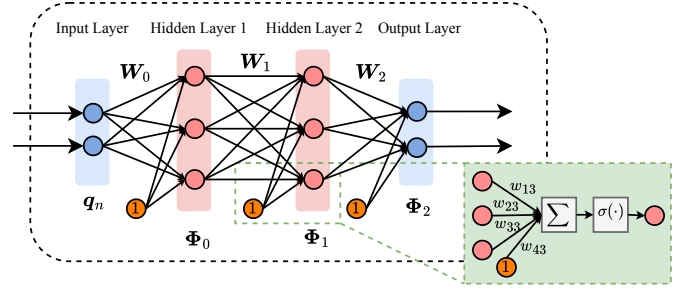


Fig. 2: Architecture of the DNN $\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta})$ with $k = 2, l_0 = 2, l_1 = l_2 = 3$, and $l_3 = 2$.

and considered a locally optimal solution, rather than a global one.

Accordingly, the ideal control input $\boldsymbol{\tau}^*$ can be approximated by the DNN with the ideal weight vector $\boldsymbol{\Phi}^* := \boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta}^*)$ as follows:

$$\boldsymbol{\tau}^* = \boldsymbol{\Phi}^* + \boldsymbol{\epsilon}, \quad (7)$$

where $\boldsymbol{\epsilon} \in \mathbb{R}^n$ is the approximation error vector, bounded by $\|\boldsymbol{\epsilon}\| \leq \bar{\epsilon}$ for some $\bar{\epsilon} \in \mathbb{R}_{>0}$. The ideal control input $\boldsymbol{\tau}^*$ is estimated online by

$$\boldsymbol{\tau} = \hat{\boldsymbol{\Phi}}, \quad (8)$$

where $\hat{\boldsymbol{\Phi}} := \boldsymbol{\Phi}(\mathbf{q}_n; \hat{\boldsymbol{\theta}})$, and $\hat{\boldsymbol{\theta}} \in \mathbb{R}^\Xi$ is the estimated weight vector for the ideal weight vector $\boldsymbol{\theta}^*$.

Substituting (7) and (8) into (6), the time derivative of V_1 can be rewritten as

$$\frac{d}{dt}V_1 \leq -\lambda_{\min}(\mathbf{K})\|\mathbf{r}\|^2 + \bar{\tau}_d\|\mathbf{r}\| + \mathbf{r}^\top (-\boldsymbol{\Phi}^* - \boldsymbol{\epsilon} + \text{sat}(\hat{\boldsymbol{\Phi}})). \quad (9)$$

Therefore, it is concluded that the filtered error $\mathbf{r}$ can be stabilized by adapting $\hat{\boldsymbol{\theta}}$ to $\boldsymbol{\theta}^*$, i.e., $\hat{\boldsymbol{\Phi}} \rightarrow \boldsymbol{\Phi}^*$, neglecting the effect of input saturation. Note that the control input saturation has not yet been considered in the derivation of the ideal control input; this will be addressed in the subsequent section.

C. Deep Neural Network (DNN) Model

The DNN architecture $\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta}) = \boldsymbol{\Phi}_k$ can be recursively represented, as follows:

$$\boldsymbol{\Phi}_i := \begin{cases} \mathbf{W}_i^\top \boldsymbol{\phi}_i(\boldsymbol{\Phi}_{i-1}), & i \in \{1, \dots, k\}, \\ \mathbf{W}_0^\top \mathbf{q}_n, & i = 0, \end{cases} \quad (10)$$

where $\mathbf{W}_i = [w_{ij}]_{i \in \{1, \dots, l_i+1\}, j \in \{1, \dots, l_{i+1}\}} \in \mathbb{R}^{(l_i+1) \times l_{i+1}}$ and $\boldsymbol{\phi}_i : \mathbb{R}^{l_i} \rightarrow \mathbb{R}^{l_i+1}$ represent the weight matrix and the activation function of the i th layer, respectively. The DNN $\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta})$ has $k+1$ layers, where the input layer is indexed by $i = 0$, and the output layer is indexed by $i = k$. For instance, in Fig. 2, the DNN $\boldsymbol{\Phi}(\mathbf{q}_n; \boldsymbol{\theta})$ with $k = 2, l_0 = 2, l_1 = l_2 = 3$, and $l_3 = 2$ is illustrated. Notice that the output size of $\boldsymbol{\Phi}(\cdot)$ is the same as that of the control input $\boldsymbol{\tau}$, i.e., $l_{k+1} = n$. The weights $\mathbf{W}_i$, $\forall i \in \{0, \dots, k\}$, are initialized randomly using uniform distribution at the beginning of the control process.

The activation function is defined as $\boldsymbol{\phi}_i(\mathbf{x}) := (\sigma(x_1), \sigma(x_2), \dots, \sigma(x_{l_i}), 1)^\top$, $\forall \mathbf{x} \in \mathbb{R}^{l_i}$, where $\sigma : \mathbb{R} \rightarrow \mathbb{R}$

is a nonlinear function, and the augmentation of 1 is used to account for bias terms in the weight matrices. In this study, the hyperbolic tangent function $\tanh(\cdot)$ was selected as the nonlinear function, i.e., $\sigma(x) = \tanh(x)$, $\forall x \in \mathbb{R}$, which provides desirable boundedness with $\|\sigma(x)\| < 1$ and $0 < \|\frac{d\sigma(x)}{dx}\| \leq 1$.

For simplicity, each layer's weights are vectorized as $\theta_i := \text{vec}(\mathbf{W}_i) \in \mathbb{R}^{\Xi_i}$, $\forall i \in \{0, \dots, k\}$, where $\Xi_i := (l_i + 1)l_{i+1}$ is the number of weights in the i th layer. The total weight vector $\theta \in \mathbb{R}^\Xi$ is defined by augmenting $\theta_i, \forall i \in \{0, \dots, k\}$, as

$$\theta := \begin{pmatrix} \theta_k \\ \theta_{k-1} \\ \vdots \\ \theta_0 \end{pmatrix} = \begin{pmatrix} \text{vec}(\mathbf{W}_k) \\ \text{vec}(\mathbf{W}_{k-1}) \\ \vdots \\ \text{vec}(\mathbf{W}_0) \end{pmatrix}, \quad (11)$$

where $\Xi := \sum_{i=0}^k \Xi_i$ represents the total number of weights in the DNN.

The gradient of $\Phi(\mathbf{q}_n; \theta)$ with respect to θ can be obtained using Proposition 1 and the chain rule as follows:

$$\frac{\partial \Phi}{\partial \theta} = \begin{bmatrix} \frac{\partial \Phi}{\partial \theta_k} & \frac{\partial \Phi}{\partial \theta_{k-1}} & \cdots & \frac{\partial \Phi}{\partial \theta_0} \end{bmatrix} \in \mathbb{R}^{n \times \Xi}, \quad (12)$$

where

$$\frac{\partial \Phi}{\partial \theta_i} = \begin{cases} (\mathbf{I}_{l_{k+1}} \otimes \phi_k^\top), & i = k, \\ \mathbf{W}_k^\top \phi'_k (\mathbf{I}_{l_k} \otimes \phi_{k-1}^\top), & i = k-1, \\ \vdots & \vdots \\ \mathbf{W}_k^\top \phi'_k \cdots \mathbf{W}_1^\top \phi'_1 (\mathbf{I}_{l_1} \otimes \mathbf{q}_n^\top), & i = 0, \end{cases} \quad (13)$$

and $\phi_i := \phi_i(\Phi_{i-1})$ and $\phi'_i := \frac{d\phi_i}{d\Phi_{i-1}}$.

In the following sections, let Φ_i^* represent the output of the i th layer with the ideal weight vector θ^* . Define $\phi_i^* = \phi_i(\Phi_{i-1}^*)$ and $\phi_i^{*'} = \frac{d\phi_i^*}{d\Phi_{i-1}^*}$. Similarly, let $\hat{\Phi}_i$ denotes the output of the i th layer with the estimated weight vector $\hat{\theta}$, and define $\hat{\phi}_i = \phi_i(\hat{\Phi}_{i-1})$ and $\hat{\phi}_i' = \frac{d\hat{\phi}_i}{d\hat{\Phi}_{i-1}}$.

IV. WEIGHT ADAPTATION LAWS

A. Weight Optimizer Design

The control objective can be represented as follows:

$$\begin{aligned} \min_{\hat{\theta}} J(\mathbf{r}; \hat{\theta}) &:= \frac{1}{2} \mathbf{r}^\top \mathbf{r}, \\ \text{subject to } c_j(\hat{\theta}) &\leq 0, \quad \forall j \in \mathcal{I}, \end{aligned} \quad (14)$$

where $J(\mathbf{r}; \hat{\theta})$ is the objective function. Inequality constraints $c_j := c_j(\hat{\theta})$, $\forall j \in \mathcal{I}$, are imposed during the weight adaptation process, where $\mathcal{I}$ denotes the set of the imposed inequality constraints. The Lagrangian function $L(\cdot)$ is defined as

$$L(\mathbf{r}, \hat{\theta}, [\lambda_j]_{j \in \mathcal{I}}) = J(\mathbf{r}; \hat{\theta}) + \sum_{j \in \mathcal{I}} \lambda_j c_j(\hat{\theta}), \quad (15)$$

where λ_j , $\forall j \in \mathcal{I}$, denotes the Lagrange multiplier for each constraint.

The adaptation laws for $\hat{\theta}$ and $[\lambda_j]_{j \in \mathcal{I}}$ are derived to solve the dual problem of (14), i.e., $\min_{\hat{\theta}} \max_{[\lambda_j]_{j \in \mathcal{I}}} L(\mathbf{r}, \hat{\theta}, [\lambda_j]_{j \in \mathcal{I}})$, as follows:

$$\frac{d}{dt} \hat{\theta} = -\alpha \frac{\partial L}{\partial \hat{\theta}} = -\alpha \left(\frac{\partial J}{\partial \hat{\theta}} + \sum_{j \in \mathcal{I}} \lambda_j \frac{\partial c_j}{\partial \hat{\theta}} \right), \quad (16a)$$

$$\frac{d}{dt} \lambda_j = \beta_j \frac{\partial L}{\partial \lambda_j} = \beta_j c_j, \quad \forall j \in \mathcal{I}, \quad (16b)$$

$$\lambda_j \leftarrow \max(\lambda_j, 0), \quad (16c)$$

where $\alpha \in \mathbb{R}_{>0}$ denotes the adaptation gain (learning rate) and $\beta_j \in \mathbb{R}_{>0}$ denotes the update rate of the Lagrange multipliers in $\mathcal{I}$, and the arguments of $L(\cdot)$ and $J(\cdot)$ are suppressed for brevity. The Lagrange multipliers associated with inequality constraints are non-negative, i.e., $\lambda_j \geq 0$, due to (16c). When a constraint c_j becomes active, i.e., $c_j > 0$, the corresponding Lagrange multiplier λ_j increases, see (16b), to address the violation, see (16a). Once the violation is resolved and the constraint is no longer active, i.e., $c_j \leq 0$, the Lagrange multiplier λ_j decreases gradually until it returns to zero, see (16c).

At steady state, where $\frac{d}{dt} \hat{\theta} = 0$ and $\frac{d}{dt} \lambda_j = 0$, the KKT conditions are satisfied, i.e., $\frac{\partial L}{\partial \hat{\theta}} = 0$, $c_j \leq 0$, $\lambda_j \geq 0$, and $\lambda_j c_j = 0$. In other words, the proposed weight optimizer updates $\hat{\theta}$ and λ_j in a way that satisfies the KKT conditions. These conditions represent the first-order necessary conditions for optimality, guiding the updates toward candidates for a locally optimal point.

B. Approximation of the Gradient of Objective Function

The adaptation law for $\hat{\theta}$ defined in (16a) involves the partial derivative of the filtered error with respect to control input $\frac{\partial \mathbf{r}}{\partial \tau}$, i.e., $\frac{\partial J}{\partial \hat{\theta}} = \left(\left(\frac{\partial \mathbf{r}}{\partial \tau} \right) \left(\frac{\partial \tau}{\partial \hat{\theta}} \right)^\top \right)^\top \mathbf{r} = \left(\left(\frac{\partial \mathbf{r}}{\partial \tau} \right) \left(\frac{\partial \hat{\Phi}}{\partial \hat{\theta}} \right)^\top \right)^\top \mathbf{r}$. Since the objective function depends on $\mathbf{r}$ of a dynamic system, obtaining $\frac{\partial \mathbf{r}}{\partial \tau}$ is not straightforward. The recommended method to calculate the exact value of $\frac{\partial \mathbf{r}}{\partial \tau}$ is to use the forward sensitivity method [21], [32] by simulating the sensitivity equation as follows:

$$\frac{d}{dt} \left(\frac{\partial \mathbf{r}}{\partial \tau} \right) = \frac{\partial}{\partial \tau} \left(\mathbf{M}^{-1} (-\mathbf{V}_m \mathbf{r} - \mathbf{K} \mathbf{r} + \mathbf{f} - \tau_d + \text{sat}(\tau)) \right). \quad (17)$$

However, this method cannot be realized, since we do not have the exact system dynamics, i.e., $\mathbf{M}, \mathbf{V}_m, \mathbf{F}$ and $\mathbf{G}$. Additionally, the computational cost of the forward sensitivity method is high, as the number of DNN's weights is generally large.

In [33], [34], the authors approximate $\frac{\partial \mathbf{r}}{\partial \tau}$ by taking the sign of each entry, i.e., $\frac{\partial \mathbf{r}}{\partial \tau} \approx \left[\text{sign} \left(\frac{\partial r_i}{\partial \tau_j} \right) \right]_{i,j \in \{1, \dots, m\}}$, assuming known control directions. However, this approach is not applicable to (5), where control directions are unknown. Instead, since the sign of the control input matrix is known—owing to the positive definiteness of $\mathbf{M}^{-1}$, see Property 1—we approximate $\frac{\partial \mathbf{r}}{\partial \tau} \approx \mathbf{I}_n$. Consequently, the adaptation law in (16a) becomes:

$$\frac{d}{dt} \hat{\theta} \approx -\alpha \left(\frac{\partial \hat{\Phi}}{\partial \hat{\theta}}^\top \mathbf{r} + \sum_{j \in \mathcal{I}} \lambda_j \frac{\partial c_j}{\partial \hat{\theta}} \right). \quad (18)$$

C. Essential and Potential Constraint Candidates

This section introduces essential constraints required to ensure the stability of the proposed CONAC, as well as conditions for potential input constraints.

First, weight norm constraints $c_{\theta_i}(\hat{\theta})$, $\forall i \in \{0, \dots, k\}$, defined in (19), are essential to prevent the weights from diverging during the adaptation process:

$$c_{\theta_i} = \frac{1}{2} \left(\|\hat{\theta}_i\|^2 - \bar{\theta}_i^2 \right), \quad (19)$$

with $\bar{\theta}_i \in \mathbb{R}_{>0}$ denoting the maximum allowable norm for $\hat{\theta}_i$, and the arguments of $c_{\theta_i}(\cdot)$ are omitted for brevity. The gradient of c_{θ_i} with respect to $\hat{\theta}$ can be obtained by accumulating the gradients of each layer as follows:

$$\frac{\partial c_{\theta_i}}{\partial \hat{\theta}_j} = \begin{cases} \hat{\theta}_i, & \text{if } i = j, \\ \mathbf{0}, & \text{if } i \neq j. \end{cases} \quad (20)$$

Next, we present the following assumptions, which specify the conditions for the potential input constraints introduced in Appendix A:

Assumption 2. The constraint function $c_j(\hat{\theta})$ is convex in the (input-) τ -space and satisfies $c_j(0) \leq 0$ and $c_j(\theta^*) \leq 0$.

Assumption 3. The selected constraints satisfy the Linear Independence Constraint Qualification (LICQ) [27, Chap. 12, Def. 12.1].

Remark 1. Assumption 2 is not restrictive, since, as aforementioned, the control input saturation is practically convex. Moreover, Assumption 3 is a standard assumption in optimization problems, ensuring that the gradients of the active constraints are linearly independent. This assumption will be used in the stability analysis (see Lemma 2).

V. STABILITY ANALYSIS

Before conducting the stability analysis, let us define the weight estimation error as $\hat{\theta} := [\hat{\theta}_i]_{i \in \{0, \dots, k\}}$, where $\hat{\theta}_i := \hat{\theta}_i - \theta_i^*$, $\forall i \in \{0, \dots, k\}$. The following lemmas are introduced for the stability analysis.

Lemma 2. If Assumptions 2 and 3 hold, then for each active constraint c_j , the angle between the output layer's weight vector $\hat{\theta}_k$ and $\frac{\partial c_j}{\partial \hat{\theta}_k}$ is positive; that is $\frac{\partial c_j}{\partial \hat{\theta}_k}^\top \hat{\theta}_k > 0$.

Proof. Since $\tau = \hat{\Phi}$, a linear mapping $T(\cdot) : \hat{\theta}_k \rightarrow \tau$, which is independent of $\hat{\theta}_k$, can be derived using Proposition 1:

$$\begin{aligned} \tau &= \hat{\Phi} = \text{vec}(\hat{\mathbf{W}}_k^\top \hat{\phi}_k) = (\mathbf{I}_{l_{k+1}} \otimes \hat{\phi}_k^\top)^\top \text{vec}(\hat{\mathbf{W}}_k) \\ &= \underbrace{(\mathbf{I}_{l_{k+1}} \otimes \hat{\phi}_k^\top)^\top}_{=: T(\hat{\phi}_k)} \hat{\theta}_k = T(\hat{\phi}_k) \hat{\theta}_k. \end{aligned} \quad (21)$$

Therefore, the convexity of the input constraints in the τ -space (see Assumption 2) is preserved in $\hat{\theta}_k$ -space, implying that $\frac{\partial c_j}{\partial \hat{\theta}_k}^\top \hat{\theta}_k > 0$. $\square$

Lemma 3. If input constraint $c_j(\hat{\theta})$, $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}$, satisfies Assumption 2, then $\|\frac{\partial c_j}{\partial \hat{\theta}_i}\|$, $\forall i \in \{k-1, \dots, 0\}$, is

bounded, provided the norms of $\hat{\theta}_i$, $\forall i \in \{k, \dots, i+1\}$, remain bounded.

For instance, if $k = 3$ and $i = 1$, $\|\frac{\partial c_j}{\partial \hat{\theta}_1}\|$ is bounded, provided that $\|\hat{\theta}_3\|$ and $\|\hat{\theta}_2\|$ are bounded.

Proof. The gradient of c_j , $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}$, with respect to $\hat{\theta}_i$ is represented as

$$\frac{\partial c_j}{\partial \hat{\theta}_i} = \frac{\partial c_j}{\partial \tau} \frac{\partial \tau}{\partial \hat{\Phi}} \frac{\partial \hat{\Phi}}{\partial \hat{\theta}_i} = \frac{\partial c_j}{\partial \tau} \mathbf{I}_n \frac{\partial \hat{\Phi}}{\partial \hat{\theta}_i}. \quad (22)$$

The boundedness of the first term $\frac{\partial c_j}{\partial \tau}$ is guaranteed, if the input argument τ is bounded owing to the convexity of c_j by Assumption 2. In addition, the boundedness of τ can be ensured by the boundedness of $\hat{\theta}_k$, since $\tau = \hat{\Phi} = (\mathbf{I}_{l_{k+1}} \otimes \hat{\phi}_k^\top) \hat{\theta}_k$ and $\|\hat{\phi}_k\|$ is bounded due to the properties of the activation functions. The third term, $\frac{\partial \hat{\Phi}}{\partial \hat{\theta}_i}$ is bounded, provided that the norms of $\hat{\theta}_i$, $\forall i \in \{k, \dots, i+1\}$, are bounded. This can be verified by using the definition of $\frac{\partial \hat{\Phi}}{\partial \hat{\theta}_i}$ given in (13). Consequently, the boundedness of $\|\frac{\partial c_j}{\partial \hat{\theta}_i}\|$, $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}$, depends on the boundedness of $\hat{\theta}_i$, $\forall i \in \{k, \dots, i+1\}$. $\square$

The following theorem states that $\mathbf{r}$ and $\hat{\theta}$ are bounded.

Theorem 1. For the dynamical system described in (1), the NAC in (8) with the weight adaptation laws in (16) ensure the boundedness of the filtered error $\mathbf{r}$ and the weight estimate $\hat{\theta}$, under the input constraints satisfying Assumption 2 and 3, provided that the weight norm constraints (19) are imposed.

Proof. Invoking the fact that the amplitude of the control input depends only on $\hat{\theta}_k$, i.e., $\tau = \hat{\Phi} = (\mathbf{I}_{l_{k+1}} \otimes \hat{\phi}_k^\top) \hat{\theta}_k$ and $\|\hat{\phi}_k\|$ is bounded, the boundedness of inner layers' weights will be established after proving the boundedness of $\mathbf{r}$ and $\hat{\theta}_k$. In addition, without loss of generality, weight norm constraints c_{θ_i} , $\forall i \in \{0, \dots, k\}$, are supposed to be active. This assumption is justified because, even if c_{θ_i} is inactive, $\hat{\theta}$ is adapted to minimize the objective function J as long as the constraint is inactive. We consider two cases: (1) the control input is saturated, and (2) the control input is not saturated.

Case 1: Control input is saturated.

As a result of the time derivative of V_1 in (9), the invariant set $\Theta_{\mathbf{r}}^1$ of $\mathbf{r}$, i.e., if $\mathbf{r}$ leaves $\Theta_{\mathbf{r}}^1$, $\frac{d}{dt} V_1$ is negative, can be obtained using Lemma 1 as follows:

$$\Theta_{\mathbf{r}}^1 = \left\{ \mathbf{r} \in \mathbb{R}^n \mid \|\mathbf{r}\| \leq \frac{\bar{\tau}_d + \bar{\tau} + \bar{\theta}_k^* \sqrt{l_k + 1} + \bar{\epsilon}}{\lambda_{\min}(\mathbf{K})} \right\}. \quad (23)$$

It is notable that the ideal DNN $\Phi^* = \mathbf{W}_k^{*\top} \phi_k^*$ is bounded by $\bar{\theta}_k^* \sqrt{l_k + 1}$. This is because, the ideal weight θ_k^* is bounded by Assumption 1 such that $\|\theta_k^*\| \leq \bar{\theta}_k^*$ for some positive constant $\bar{\theta}_k^*$. Hence, invoking $\|\mathbf{W}_k^*\|_F = \|\theta_k^*\| \leq \bar{\theta}_k^*$ and $\|\phi_k^*\| \leq \sqrt{l_k + 1}$, we have $\|\Phi^*\| \leq \bar{\theta}_k^* \sqrt{l_k + 1}$.

To investigate the boundedness of $\hat{\theta}_k$, consider the Lyapunov function candidate $V_2 := \frac{1}{2\alpha} \hat{\theta}_k^\top \hat{\theta}_k$. Taking the time derivative of V_2 yields:

$$\begin{aligned} \frac{d}{dt} V_2 &= -\hat{\theta}_k^\top \left((I_{l_{k+1}} \otimes \hat{\phi}_k^\top)^\top \mathbf{r} + \sum_{j \in \mathcal{I}} \lambda_j \frac{\partial c_j}{\partial \hat{\theta}_k} \right) \\ &= -\hat{\theta}_k^\top \left((I_{l_{k+1}} \otimes \hat{\phi}_k^\top)^\top \mathbf{r} + \lambda_{\theta_k} \hat{\theta}_k \right) \\ &\quad + \sum_{j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}} \lambda_j \frac{\partial c_j}{\partial \hat{\theta}_k} \\ &\leq -\lambda_{\theta_k} \|\hat{\theta}_k\|^2 + \underbrace{\|(I_{l_{k+1}} \otimes \hat{\phi}_k^\top)\| \|\mathbf{r}\|}_{=: \gamma_1 \in \mathbb{R}_{\geq 0}} \|\hat{\theta}_k\| \\ &\quad - \underbrace{\sum_{j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}} \lambda_j \hat{\theta}_k^\top \frac{\partial c_j}{\partial \hat{\theta}_k}}_{=: \gamma_2 \in \mathbb{R}_{\geq 0}, \text{ by Lemma 2 and Assumption 3}} \\ &\leq -\lambda_{\theta_k} \|\hat{\theta}_k\|^2 + \gamma_1 \|\mathbf{r}\| \|\hat{\theta}_k\|. \end{aligned} \quad (24)$$

According to (24) and Lemma 1, the invariant set of $\hat{\theta}_k$ is defined as

$$\Theta_{\theta_k}^1 = \left\{ \hat{\theta}_k \in \mathbb{R}^{\Xi_k} \mid \|\hat{\theta}_k\| \leq \frac{\gamma_1 (\bar{\tau}_d + \bar{\tau} + \bar{\theta}_k^* \sqrt{l_k + 1} + \bar{\epsilon})}{\lambda_{\theta_k} \lambda_{\min}(\mathbf{K})} \right\}. \quad (25)$$

The satisfaction of the constraints, i.e., c_j , $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k-1\}}$, can be verified through the boundedness of $\hat{\theta}_k$. For the output layer's weight norm constraint c_{θ_k} , if the corresponding Lagrange multiplier λ_{θ_k} increases sufficiently large due to the constraint violation, $\hat{\theta}_k$ approaches toward the origin as the invariant set $\Theta_{\theta_k}^1$ shrinks to a point. The remaining input constraints can be validated implicitly through the term γ_2 in (24). Similarly to c_{θ_k} , when an input constraint c_j , $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}$, is violated, the corresponding Lagrange multiplier λ_j increases sufficiently large, leading to an increase in γ_2 . It makes γ_2 dominates the right-hand side of (24), which leads $\frac{d}{dt} V_2$ negative definite. This drives $\hat{\theta}_k$ toward the origin until the constraint is satisfied.

Therefore, the boundedness of $\mathbf{r}$ and $\hat{\theta}_k$ is guaranteed by the invariant sets $\Theta_{\mathbf{r}}^1$ and $\Theta_{\theta_k}^1$, and the satisfactions of c_j , $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k-1\}}$, is ensured.

Case 2: Control input is not saturated.

Since the input constraints are inactive, the saturation function $\text{sat}(\cdot)$ in (9) and the input constraint functions c_j , $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}$, in (24) can be neglected. Consider the Lyapunov function candidate $V_3 := V_1 + V_2$, whose time derivative is given by:

$$\begin{aligned} \frac{d}{dt} V_3 &= -\lambda_{\min}(\mathbf{K}) \|\mathbf{r}\|^2 + \bar{\tau}_d \|\mathbf{r}\| + \mathbf{r}^\top (\hat{\Phi} - \Phi^* - \epsilon) \\ &\quad - \hat{\theta}_k^\top \left((I_{l_{k+1}} \otimes \hat{\phi}_k^\top)^\top \mathbf{r} + \lambda_{\theta_k} \hat{\theta}_k \right) \\ &\leq -\lambda_{\min}(\mathbf{K}) \|\mathbf{r}\|^2 + \mathbf{r}^\top \hat{\Phi} + (\bar{\tau}_d + \|\Phi^* + \epsilon\|) \|\mathbf{r}\| \\ &\quad - \hat{\Phi}^\top \mathbf{r} - \lambda_{\theta_k} \|\hat{\theta}_k\|^2 \\ &\leq -\lambda_{\min}(\mathbf{K}) \|\mathbf{r}\|^2 + \left(\bar{\tau}_d + \bar{\theta}_k^* \sqrt{l_k + 1} + \bar{\epsilon} \right) \|\mathbf{r}\| \\ &\quad - \lambda_{\theta_k} \|\hat{\theta}_k\|^2. \end{aligned} \quad (26)$$

Based on the same reasoning as in Case 1, the boundedness of $\mathbf{r}$ is ensured by the invariant set:

$$\Theta_{\mathbf{r}}^2 = \left\{ \mathbf{r} \in \mathbb{R}^n \mid \|\mathbf{r}\| \leq \frac{\bar{\tau}_d + \bar{\theta}_k^* \sqrt{l_k + 1} + \bar{\epsilon}}{\lambda_{\min}(\mathbf{K})} \right\}. \quad (27)$$

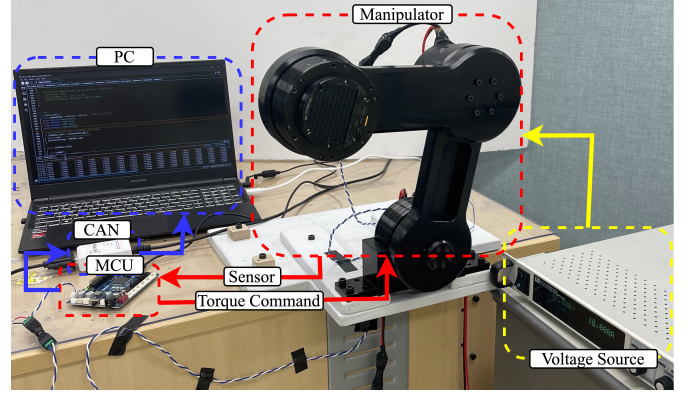


Fig. 3: Experimental setup of the two-link robotic manipulator used for real-time validation.

The boundedness of $\hat{\theta}_k$ also can be ensured by the invariant set $\Theta_{\theta_k}^2 = \{\hat{\theta}_k \in \mathbb{R}^{\Xi_k} \mid \|\hat{\theta}_k\| \leq \bar{\theta}_k\}$, as λ_{θ_k} remains positive unless the weight norm constraint c_{θ_k} is satisfied.

The boundedness of the inner layer weights and the satisfaction of their corresponding constraints c_j , $\forall j \in \{\theta_i\}_{i \in \{k-1, \dots, 0\}}$, can be established recursively using [35, Chap. 4, Thm. 1.9]. The dynamics of $\hat{\theta}_i$, $\forall i \in \{k-1, \dots, 0\}$, are represented as

$$\frac{d}{dt} \hat{\theta}_i = -\alpha \left(\frac{\partial \hat{\Phi}}{\partial \hat{\theta}_i}^\top \mathbf{r} + \lambda_{\theta_i} \hat{\theta}_i + \sum_{j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}} \lambda_j \frac{\partial c_j}{\partial \hat{\theta}_i} \right). \quad (28)$$

By Lemma 3, $\hat{\theta}_i$ is bounded, provided that $\hat{\theta}_i$, $\forall i \in \{k, \dots, i+1\}$, are bounded. This holds because the system matrix $-\lambda_{\theta_i} \mathbf{I}_{\Xi_i}$ is stable, and the residual terms—such as $\frac{\partial \hat{\Phi}}{\partial \hat{\theta}_i}^\top \mathbf{r}$, and $\frac{\partial c_j}{\partial \hat{\theta}_i}$ —are bounded. Moreover, the Lagrange multiplier λ_j does not diverge since the input constraints c_j , $\forall j \in \mathcal{I} \setminus \{\theta_i\}_{i \in \{0, \dots, k\}}$ are satisfied before divergence can occur. Therefore, starting from the $(k-1)$ th layer, the boundedness of $\hat{\theta}_i$ can be established recursively down to the input layer ($i = 0$). Moreover the satisfaction of the weight norm constraints for the inner layers can be verified, as the Lagrange multipliers λ_{θ_i} , $\forall i \in \{k-1, \dots, 0\}$, increase sufficiently large to stabilize the $\hat{\theta}_i$ dynamics in (28), leading to its convergence toward the origin.

In conclusion, the filtered tracking error $\mathbf{r}$ and the weight estimate $\hat{\theta}$ are bounded, and all imposed constraints c_j , $\forall j \in \mathcal{I}$, are satisfied. $\square$

VI. REAL-TIME IMPLEMENTATION AND VALIDATION

A. Validation Setup

To validate the effectiveness of the proposed CONAC, a real-time experiment was conducted on a two-link robotic manipulator, as shown in Fig. 3. The OpenCR1.0 board [36] with a 32-bit ARM Cortex and a 216 MHz clock frequency was used for the real-time implementation. The two-link robotic manipulator actuated by two Cubemars AK10-90 servo motors. The servos were controlled using the OpenCR1.0

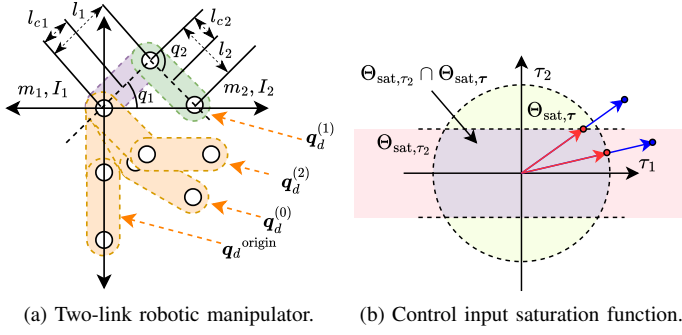


Fig. 4: Two-link robotic manipulator and control saturation function.

TABLE I: Two-link robotic manipulator's parameters.

Symbol	Description	Value
m_1, m_2	Mass	2.465 kg
l_1, l_2	Length	0.2 m
l_{c1}, l_{c2}	Center of mass	0.139 m
I_1, I_2	Inertia	0.069 kg · m ²

board which transmits the torque reference to the corresponding servo via controller area network (CAN) communication. The system dynamics of the two-link robotic manipulator can follow the Euler-Lagrange equation (1), where $\mathbf{q}$ denotes joint angles and $\boldsymbol{\tau}$ denotes the control input (joint torques), and is illustrated in Fig. 4a. The measured and/or estimated system parameters are provided in Table I.

The desired trajectory is designed with 4 segments as follows:

$$\mathbf{q}_d(t) = \begin{pmatrix} q_{d1} \\ q_{d2} \end{pmatrix} = \begin{cases} \mathcal{P}^5(t; \mathbf{q}_d^{(0)}, \mathbf{q}_d^{(1)}, t_f), & \text{if } 0 \leq t < t_f, \\ \mathcal{P}^5(t; \mathbf{q}_d^{(1)}, \mathbf{q}_d^{(2)}, t_f), & \text{if } t_f \leq t < 2t_f, \\ \mathcal{P}^5(t; \mathbf{q}_d^{(2)}, \mathbf{q}_d^{(1)}, t_f), & \text{if } 2t_f \leq t < 3t_f, \\ \mathcal{P}^5(t; \mathbf{q}_d^{(1)}, \mathbf{q}_d^{(0)}, t_f), & \text{if } 3t_f \leq t < 4t_f, \end{cases} \quad (29)$$

where $\mathcal{P}^5(t; \mathbf{x}_1, \mathbf{x}_2, t_f)$ denotes a quintic (5th-order) polynomial trajectory that moves the system from $\mathbf{x}_1 \in \mathbb{R}^2$ to $\mathbf{x}_2 \in \mathbb{R}^2$ in t_f seconds, with zero initial and final velocities. The initial and final joint angles of the segments are defined as follows: $\mathbf{q}_d^{(0)} = (-\frac{1}{3}\pi, \frac{1}{3}\pi)^\top$, $\mathbf{q}_d^{(1)} = (\frac{1}{4}\pi, -\frac{1}{2}\pi)^\top$, $\mathbf{q}_d^{(2)} = (-\frac{1}{4}\pi, \frac{1}{4}\pi)^\top$, as illustrated in Fig. 4a. The time duration of the segments is defined as $t_f = 3$ s. To demonstrate the learning capability of the DNN, the desired trajectory was applied twice. The first and second repetitions of the desired trajectory are referred to as Episode 1 and Episode 2, respectively.

However, due to the random initialization of the DNN weights, CONAC's initial performance is expected to be poor, leading to unstable behavior. Therefore, in order to avoid the instability caused by the initial poor performance of CONAC, a warm-up phase was introduced, prior to applying the desired trajectory. In the warm-up phase, the manipulator was initially positioned at a stable equilibrium point, denoted as $\mathbf{q}_d^{\text{origin}} = (-\frac{1}{2}\pi, 0)^\top$. Subsequently, a trajectory $\mathcal{P}^5(t; \mathbf{q}_d^{\text{origin}}, \mathbf{q}_d^{(0)}, 3)$ was used to move the manipulator from $\mathbf{q}_d^{\text{origin}}$ to $\mathbf{q}_d^{(0)}$ over a duration of 3 seconds. Once the manipulator reached $\mathbf{q}_d^{(0)}$,

TABLE II: Properties of the controllers

Description	Input Constraint Handling
CONAC with (C ₁) large update rate β_j . ($\beta_{\bar{\tau}_2} = \beta_{\tau_2} = 100$ and $\beta_\tau = 10$)	(32) $c_{\bar{\tau}_2}$ and c_{τ_2} with $\bar{\tau}_2 = -\tau_2 = 3.5$, and (34) c_τ with $\bar{\tau} = 11$.
CONAC with (C ₂) small update rate β_j . ($\beta_{\bar{\tau}_2} = \beta_{\tau_2} = 1$ and $\beta_\tau = 0.1$)	(32) $c_{\bar{\tau}_2}$ and c_{τ_2} with $\bar{\tau}_2 = -\tau_2 = 3.5$, and (34) c_τ with $\bar{\tau} = 11$.
CONAC without input constraints. (C ₃) ($\beta_{\bar{\tau}_2} = \beta_{\tau_2} = \beta_\tau = 0$)	No input constraints are imposed.
NAC with auxiliary system. (C ₄) (conventional method)	$\bar{\tau}_1 = -\tau_1 = 10.428$ $\bar{\tau}_2 = -\tau_2 = 3.5$.

a constant desired trajectory $\mathbf{q}_d = \mathbf{q}_d^{(0)}$ was maintained for 8 seconds to allow the system to settle and eliminate the influence of transient dynamics.

In the validation, we assume that the actuators are subject to a convex saturation function that projects the control input onto the convex set $\Theta_{\text{sat}} := \{\Theta_{\text{sat}, \tau_2} \cap \Theta_{\text{sat}, \tau}\}$, as illustrated in Fig. 4b, where

$$\Theta_{\text{sat}, \tau_2} := \{\tau_2 \mid |\tau_2| \leq 3.5\}, \quad (30a)$$

$$\Theta_{\text{sat}, \tau} := \{\boldsymbol{\tau} \mid \|\boldsymbol{\tau}\| \leq 11\}. \quad (30b)$$

The sets defined in (30a) and (30b) can be physically interpreted as the current limit imposed by the torque limit of the second joint actuator and the power source, respectively. The corresponding input constraints are mathematically described in Appendix A as the input bound constraint (32) and the input norm constraint (34), respectively.

For comparative analysis, four controllers were implemented, with their properties summarized in Table II.

Controller 1 (C₁)—CONAC with a large update rate β_j , $\forall j \in \mathcal{I}$, for the Lagrange Multipliers: To handle input saturation, the proposed CONAC was implemented with the input bound constraints $c_{\bar{\tau}_2}$ and c_{τ_2} (32) for the second link, and the input norm constraint c_τ (34), as follows:

$$c_{\bar{\tau}_2} = \tau_2 - \bar{\tau}_2, \quad c_{\tau_2} = \tau_2 - \bar{\tau}_2, \quad c_\tau = \frac{1}{2} (\|\boldsymbol{\tau}\| - \bar{\tau})^2, \quad (31)$$

where $\bar{\tau}_2 = -\tau_2 = 3.5$, and $\bar{\tau} = 11$. Therefore, controller (C₁) utilizes the full capability of the actuator, see (30a) and (30b). The weight norm constraints $c_{\theta_i}(\cdot)$, $\forall i \in \{0, \dots, k\}$, defined in (19), were also incorporated into CONAC framework with $\bar{\theta}_0 = \bar{\theta}_1 = \bar{\theta}_2 = 6$. As described in (8), the control input (torque) corresponds to the output of the DNN, and the adaptation law is defined in (16).

The update rates of the Lagrange multipliers for the weight norm constraints were set to $\beta_{\theta_i} = 1$, $\forall i \in \{0, \dots, k\}$. For the input constraints, large update rates were used: $\beta_{\bar{\tau}_2} = \beta_{\tau_2} = 100$ and $\beta_\tau = 10$, in order to investigate the effect of large update rates.

Controller 2 (C₂)—CONAC with a small update rate β_j , $\forall j \in \mathcal{I}$, for the Lagrange Multipliers: The proposed CONAC was implemented with the same input constraints as in (C₁), but with update rates for the input constraints reduced by a factor of 100: $\beta_{\bar{\tau}_2} = \beta_{\tau_2} = 1$ and $\beta_\tau = 0.1$. All other properties and parameters were kept identical to those of (C₁).

Controller 3 (C₃)—CONAC without input constraints: The third controller was identical to (C₁) and (C₂), except that the input constraints were removed to highlight the constraint handling capability of the proposed CONAC. In this case, the update rates of the Lagrange multipliers associated with the input constraints, i.e., $\beta_{\bar{\tau}_2}, \beta_{\tau_2}$ and β_{τ} , were set to zero, such that the corresponding Lagrange multipliers $[\lambda_j]_{j \in \{\bar{\tau}_2, \tau_2, \tau\}}$ were not updated, see (16b). All other properties and parameters were kept the same as in (C₁) and (C₂).

Controller 4 (C₄)—NAC with auxiliary system (conventional method): As a baseline for comparison, the auxiliary system-based approach from [12], [13], [26] was implemented. The auxiliary system is defined by $\frac{d}{dt}\zeta = A_\zeta \zeta + B_\zeta \Delta\tau$, with initial condition $\zeta|_{t=0} = \mathbf{0}_{2 \times 1}$, where $\zeta \in \mathbb{R}^2$ represents the auxiliary state variables, and $A_\zeta \in \mathbb{R}_{<0}^{2 \times 2}$ and $B_\zeta \in \mathbb{R}^{2 \times 2}$ are user-designed matrices. The auxiliary state ζ is driven by the overflow amount of the control input $\Delta\tau$, where each element is defined as $\Delta\tau_i = \tau_i - \max(\underline{\tau}_i, \min(\bar{\tau}_i, \tau_i))$, $\forall i \in \{1, 2\}$.

This auxiliary state was incorporated into the adaptation law by replacing the filtered tracking error r with $r + \zeta$. As a result, the weights were adapted to minimize both the filtered tracking error and the auxiliary state, thereby mitigating saturation in each control input.

The auxiliary system matrices were chosen as $A_\zeta = \text{diag}(-10, -10)$ and $B_\zeta = \text{diag}(10^3, 10^3)$, to balance responsiveness to input overflow with the convergence speed of the auxiliary dynamics. Since the auxiliary system only accounts for element-wise input saturation—corresponding to the input bound constraint (32)—the upper bound of the first actuator torque $\bar{\tau}_1 = -\underline{\tau}_1$ was set to 10.428 to approximate the input norm constraint $\bar{\tau} = 11$, given $\bar{\tau}_1^2 + \bar{\tau}_2^2 = \bar{\tau}^2$.

All other properties and parameters were kept the same as in (C₁), (C₂) and (C₃) expect that the weight boundedness was handled using the projection operator used in [11].

For all controllers, i.e., (C₁), (C₂), (C₃), and (C₄), the input vector to the DNN was defined as $q_n = (q^\top, \frac{d}{dt}q^\top, r^\top, 1)^\top \in \mathbb{R}^7$, where the scalar 1 was included to account for the bias term in the weight matrix. All DNNs shared the same architecture, consisting of two hidden layers with four nodes each, i.e., $k = 2$, and $l_0 = 6, l_1 = 4, l_2 = 4$, and $l_3 = 2$ in (10). The adaptation gain was set to $\alpha = 0.2$, and the filtering matrix in (4) was chosen as $\Lambda = \text{diag}(5, 15)$. To support real-time implementation, the controller operated at a sampling rate of 250 Hz, with control results collected via CAN communication using a PCAN-USB interface and recorded on a computer at the same rate.

The tracking performance was quantitatively evaluated using the L_2 -norm of the each joint angle's tracking error, defined as $\sqrt{\int_0^T \|e_i(t)\|^2 dt}$, $\forall i \in \{1, 2\}$, where $T \in \mathbb{R}_{>0}$ denotes the duration of each episode.

B. Validation Results

1) *Tracking Performance:* The real-time implementation results are presented in Fig. 5, and the quantitative comparison of tracking performance is summarized in Table III. Across

TABLE III: Quantitative comparison of performances' L_2 norm.

Episode 1				
	(C ₁)	(C ₂)	(C ₃)	(C ₄)
$e_1 \times 10^3$ / rad	25.130	21.719	23.168	24.195
$e_2 \times 10^3$ / rad	25.016	10.530	13.061	12.927
Episode 2				
	(C ₁)	(C ₂)	(C ₃)	(C ₄)
$e_1 \times 10^3$ / rad	7.066 (−71.6%)	8.021 (−63.1%)	8.408 (−63.7%)	19.721 (−18.5%)
$e_2 \times 10^3$ / rad	2.198 (−88.4%)	2.435 (−76.9%)	3.554 (−72.8%)	4.256 (−67.1%)

the two episodes, all controllers improved their tracking performance. Specifically, the L_2 -norm of e_1 was reduced by 71.6% for (C₁), 63.1% for (C₂), 63.7% for (C₃), and 18.5% for (C₄); while the L_2 -norm of e_2 was reduced by 88.4%, 76.9%, 72.8%, and 67.1%, respectively. Qualitatively, as illustrated in Fig. 5a and Fig. 5b, oscillations were suppressed during the second episode. These results indicate that the DNNs in all controllers successfully learned the ideal control input τ^* using the filtered tracking error r , despite τ^* being completely unknown a priori. Furthermore, the stable learning observed under random weight initialization and without prior knowledge of the system, demonstrated the effectiveness of the proposed CONAC in enabling online learning capability.

Notably, in both episodes, all controllers failed to track the desired trajectory of q_1 during certain time intervals: from 17.5 s to 19.9 s in Episode 1, and from 23.5 s to 25.9 s and from 29.5 s to 31.9 s in Episode 2, as shown in Fig. 5a. This degradation in tracking performance was attributed to actuator saturation during those intervals, as observed in Fig. 5d and Fig. 5e.

The tracking performance of CONAC with input constraints—i.e., (C₁) and (C₂)—outperformed the auxiliary system-based NAC (C₄) in the second episode, as shown in Table III. This improvement was attributed to the ability of (C₁) and (C₂) to fully utilize the actuator's capacity, whereas (C₄) was limited by the design of the auxiliary system, which constrained the first joint actuator. While (C₃) also showed better performance than (C₄), this was primarily because it did not consider input saturation and thus used more of the actuator's capacity. However, since (C₃) did not handle the control input saturation, it generated excessively large control efforts to minimize the tracking error, even though these efforts could not be realized due to actuator limits, as shown in Fig. 5c–5e. In addition, the severity of saturation violations in (C₃) increased during periods when the input constraints were active. These large violations pose a significant risk to the physical system, potentially causing actuator damage or failure. Furthermore, once the control saturation was lifted—typically due to changes in the desired trajectory—this resulted in oscillatory behavior, as CONAC's weights had been adapted to generate excessively high control inputs, see Fig. 5e, where the input constraints were lifted at 25.9 s and 31.9 s.

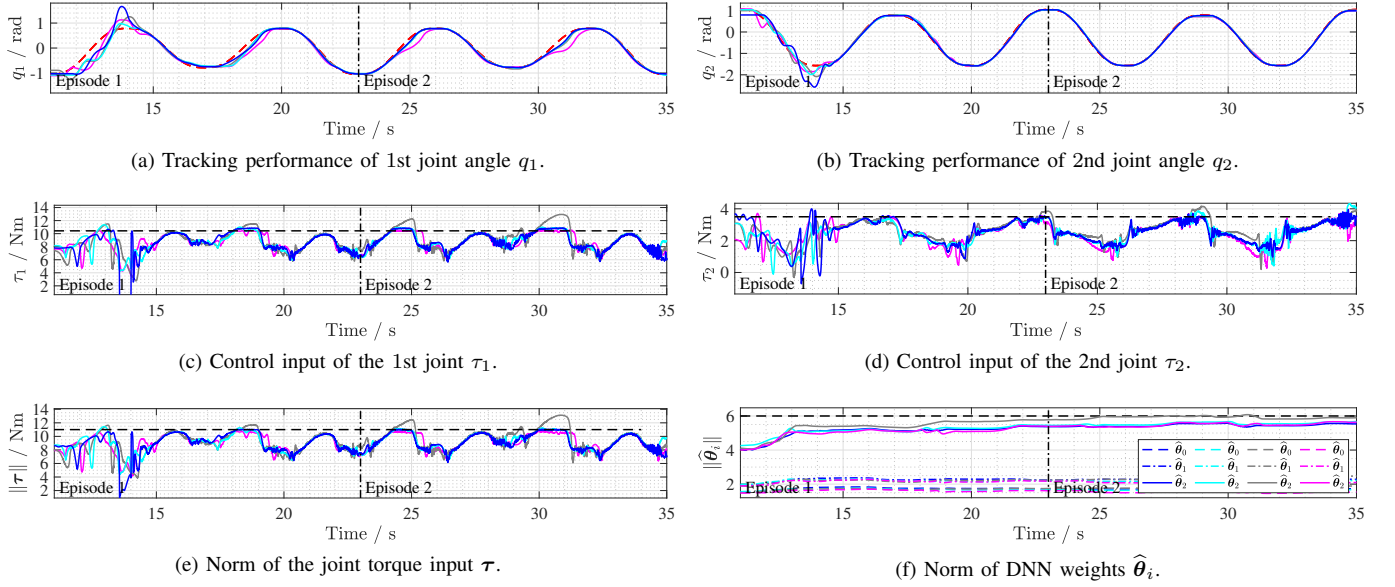


Fig. 5: Real-time implementation results of (C₁) [—], (C₂) [—], (C₃) [—], (C₄) [—], and desired trajectory q_d [---].

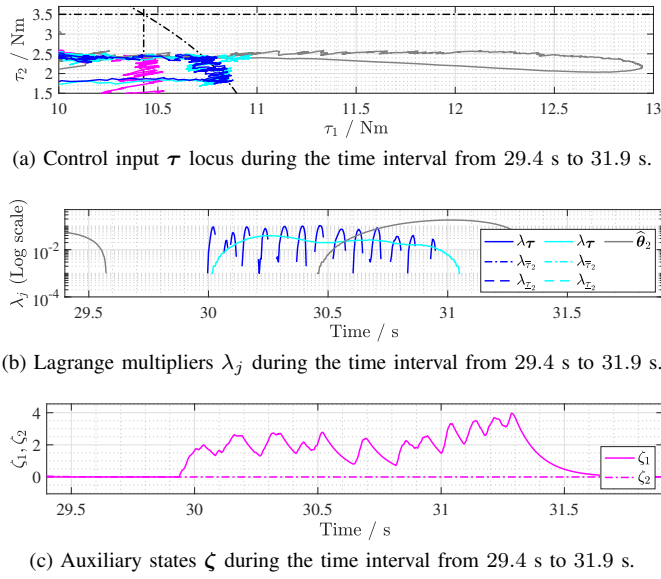


Fig. 6: Constraint handling behavior of (C₁) [—], (C₂) [—], (C₃) [—], and (C₄) [—] during the time interval from 29.4 s to 31.9 s.

2) *Constraint Handling*: In this section, the constraint handling capability of all controllers are investigated. As shown in Fig. 5c–5e, all controllers except (C₃), which did not consider the control input saturation, satisfied the control input saturation illustrated in Fig. 4b. In (C₁) and (C₂), control input saturation was addressed through corresponding input constraints within the constrained optimization framework, whereas (C₄) employed an auxiliary system, as summarized in Table II.

To investigate the constraint handling process, the time interval from 29.4 s to 31.9 s was selected, during which the control input saturation was active for all controllers. In

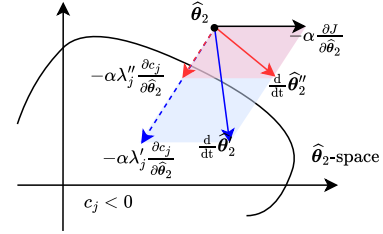


Fig. 7: The effect of the Lagrange multiplier λ_j on the adaptation direction of $\hat{\theta}_2$ is illustrated. The notations $(\cdot)'$ and $(\cdot)''$ denote two distinct cases corresponding to large and small values of the Lagrange multiplier, respectively, i.e., $\lambda_j' > \lambda_j''$.

Fig. 6, the control input locus, the Lagrange multipliers, and the auxiliary states are illustrated over this time interval. The properties of the controllers are illustrated in Fig. 6a, where the control input loci of (C₁) and (C₂) fully explore the feasible input domain shown in Fig. 4b, whereas (C₃) exceeds the input saturation limits, and (C₄) is restricted in its first actuator input limit due to the auxiliary system design.

The input constraint handling between 29.4 s to 31.9 s is examined in detail. For (C₁) and (C₂), activation of input constraints c_j dynamically generated the corresponding Lagrange multipliers λ_j , as shown in Fig. 6b. These multipliers guided the adaptation of the weights $\hat{\theta}_i$, $\forall i \in \{0, 1, 2\}$, to reduce constraint violations in accordance with (16a).

To further illustrate this effect, Fig. 7 compares the adaptation direction of the output layer's weight $\hat{\theta}_2$ under two different magnitudes of λ_j . Under Assumptions 2 and 3, the convexity of c_j in the $\hat{\theta}_2$ -space (see Lemma 2) is illustrated in Fig. 7. The figure highlights the influence of the update rate β_j : a larger β_j (as in (C₁)) yields a larger λ_j , resulting in a more aggressive adjustment of $\hat{\theta}_2$ via the term $-\lambda_j \frac{\partial c_j}{\partial \hat{\theta}_2}$ in (16a), thereby accelerating constraint satisfaction—though at the cost

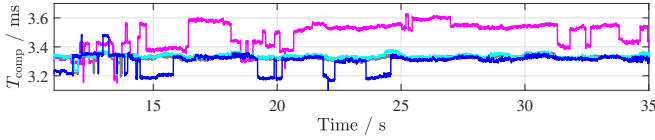


Fig. 8: Computational time T_{comp} of (C1) [—], (C2) [—], (C3) [—], and (C4) [—].

of oscillatory learning behavior. In contrast, the smaller update rate in (C2) leads to more conservative weight updates and smoother, but slower, convergence. These behaviors are clearly observed in Fig. 6a and Fig. 6b, where (C1) satisfies the input constraint c_{τ} more quickly than (C2), which shows repeated growth and reset of λ_{τ} .

For (C4), the auxiliary state ζ_1 was generated in response to the control input saturation of the first link actuator τ_1 , as shown in Fig. 6c. Since the weights were adapted to reduce both the filtered error r and the auxiliary state ζ , the control input saturation was subsequently reduced, as evident in Fig. 6a. Once the saturation was resolved at around 31.2 s (see Fig. 5c), the auxiliary state ζ_1 converged to zero due to the stable state matrix A_{ζ} in the auxiliary system dynamics, given by $\frac{d}{dt}\zeta = A_{\zeta}\zeta + B_{\zeta}\Delta\tau$ with $\Delta\tau = \mathbf{0}_{2 \times 1}$, as shown again in Fig. 6c.

Finally, the handling of the weight norm constraints c_{θ_i} , $\forall i \in \{0, 1, 2\}$, is investigated. Among all controllers, the weight norm constraint was only activated for the output layer weights $\hat{\theta}_2$ in (C3), as shown in Fig. 5f. This is because the weights in (C1), (C2), and (C4) were implicitly suppressed by the input constraints, which were not applied in (C3). Furthermore, the constraints on the inner and hidden layer weights were never violated; that is, both $\|\hat{\theta}_0\|$ and $\|\hat{\theta}_1\|$ consistently remained below their limits. This was attributed to the fact that the backpropagated error being scaled by the elements' gradients of the activation function, which is bounded in $(0, 1]$, i.e., $\sigma(\cdot) = \tanh(\cdot)$.

Additional details on the constraint handling process are provided in Fig. 6b. Similar to the case of the input constraints, the Lagrange multipliers λ_{θ_2} increased at around 30.4 s, as shown in Fig. 6b, when $\hat{\theta}_2$ exceeded the prescribed norm constraint $\bar{\theta}_2$ (see Fig. 5f), thereby adjusting the adaptation direction of $\hat{\theta}_2$ to mitigate the violation of the constraint c_{θ_2} .

3) *Effectiveness of CONAC in Real-time Applications:* The effectiveness of the proposed CONAC in real-time applications was evaluated by measuring the computational time using CPU clock ticks on the OpenCR1.0 board. As shown in Fig. 8, the computational times of all controllers remained below 4 ms, demonstrating that the proposed CONAC is suitable for real-time implementation at a sampling rate of 250 Hz.

VII. CONCLUSION

This paper proposed a constrained optimization-based neuro-adaptive controller (CONAC) for unknown Euler-Lagrange systems, addressing both weight norm and convex input constraints within a unified optimization framework. The

stability of CONAC was analyzed using Lyapunov theory, demonstrating bounded tracking errors and weight estimation under real-time adaptation.

CONAC effectively incorporated both convex input constraints and weight norm constraints, ensuring that actuator limitations and neural network weights remained within predefined bounds. By embedding these constraints into the optimization process, CONAC ensured weight convergence in accordance with the Karush-Kuhn-Tucker (KKT) conditions, thereby ensuring both stability and optimality.

Real-time implementation validated the superior performance of CONAC compared to a conventional method. CONAC successfully handled complex input constraints while rigorously managing weight norms, resulting in improved tracking accuracy and stable behavior without significant oscillations.

Future work may extend this approach to include constraints on both control inputs and system states, further enhancing the flexibility and robustness of neuro-adaptive control systems within constrained optimization frameworks.

APPENDIX

A. Input Constraint Candidates

This section introduces potential input constraints that can be incorporated into the proposed CONAC framework.

1) *Input Bound Constraint:* Most physical systems are subject to control input limitations due to inherent electrical and mechanical constraints. These are expressed as $c_{\tau} := [c_{\tau_i}]_{i \in \{1, \dots, n\}}$ and $c_{\tau} := [c_{\tau_i}]_{i \in \{1, \dots, n\}}$, where

$$c_{\tau_i} = \tau_{(i)} - \tau_{\bar{i}}, \quad c_{\tau_i} = \tau_{\bar{i}} - \tau_{(i)}, \quad (32)$$

with $\tau_{\bar{i}} \in \mathbb{R}$ and $\tau_{\bar{i}} \in \mathbb{R}$ representing the maximum and minimum control input bounds, respectively. The gradients of c_{τ} and c_{τ} with respect to $\hat{\theta}$ are given by

$$\begin{aligned} \frac{\partial c_{\tau}}{\partial \hat{\theta}} &= + \frac{\partial \hat{\Phi}}{\partial \hat{\theta}} = + \begin{bmatrix} (\mathbf{I}_{l_{k+1}} \otimes \hat{\Phi}_k^{\top}) & \dots & (\cdot) \end{bmatrix} \in \mathbb{R}^{n \times \Xi}, \\ \frac{\partial c_{\tau}}{\partial \hat{\theta}} &= - \frac{\partial \hat{\Phi}}{\partial \hat{\theta}} = - \begin{bmatrix} (\mathbf{I}_{l_{k+1}} \otimes \hat{\Phi}_k^{\top}) & \dots & (\cdot) \end{bmatrix} \in \mathbb{R}^{n \times \Xi}. \end{aligned} \quad (33)$$

2) *Input Norm Constraint:* Consider the control input τ as the torque corresponding to its generalized coordinate. Since torque is typically linearly proportional to current, actuators that share a common power source are often subject to total current limitations. This can be captured by the following inequality constraint:

$$c_{\tau} = \frac{1}{2} (\|\tau\|^2 - \bar{\tau}^2), \quad (34)$$

with the maximum allowable control input magnitude $\bar{\tau} \in \mathbb{R}_{>0}$. This input norm constraint is also commonly applied in current and torque control problems for electric motors. The gradient of c_{τ} with respect to $\hat{\theta}$ are given by

$$\frac{\partial c_{\tau}}{\partial \hat{\theta}} = \sum_{i=1}^n \tau_i \left(\text{row}_i \left(\frac{\partial \hat{\Phi}}{\partial \hat{\theta}} \right) \right)^{\top} = \frac{\partial \hat{\Phi}}{\partial \hat{\theta}}^{\top} \tau \in \mathbb{R}^{\Xi}. \quad (35)$$

It should be noted that constraints (32) and (34) can be imposed simultaneously, as their gradients (33) and (35) are linearly independent, satisfying Assumption 3, i.e., the LICQ condition.

REFERENCES

- [1] D. S. Bernstein, *Matrix Mathematics: Theory, Facts, and Formulas (Second Edition)*. Princeton University Press, 2009.
- [2] P. Ioannou and B. Fidan, *Adaptive Control Tutorial*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006.
- [3] G. Tao, *Adaptive Control Design and Analysis (Adaptive and Learning Systems for Signal Processing, Communications and Control Series)*. USA: John Wiley & Sons, Inc., 2003.
- [4] F. L. Lewis, A. Yesildirak, and S. Jagannathan, *Neural Network Control of Robot Manipulators and Nonlinear Systems*. USA: Taylor & Francis, Inc., 1998.
- [5] J. A. Farrell and M. M. Polycarpou, *Adaptive Approximation Based Control: Unifying Neural, Fuzzy and Traditional Adaptive Approximation Approaches (Adaptive and Learning Systems for Signal Processing, Communications and Control Series)*. USA: Wiley-Interscience, 2006.
- [6] F. Lewis, A. Yesildirek, and K. Liu, "Multilayer neural-net robot controller with guaranteed tracking performance," *IEEE Trans. Neural Netw.*, vol. 7, no. 2, pp. 388–399, 1996.
- [7] S. S. Ge, C. C. Hang, T. H. Lee, and Z. Tao, *Stable adaptive neural network control*, ser. The International Series on Asian Studies in Computer and Information Science. New York, NY: Springer, Dec. 2010.
- [8] A. Yeşildirek and F. Lewis, "Feedback linearization using neural networks," *Automatica*, vol. 31, no. 11, pp. 1659–1664, 1995.
- [9] J. Liu, *Radial basis function (RBF) neural network control for mechanical systems*, 2013th ed. Berlin, Germany: Springer, Jan. 2013.
- [10] S. Ge and C. Wang, "Direct adaptive NN control of a class of nonlinear systems," *IEEE Trans. Neural Netw.*, vol. 13, no. 1, pp. 214–221, 2002.
- [11] O. S. Patil, D. M. Le, M. L. Greene, and W. E. Dixon, "Lyapunov-derived control and adaptive update laws for inner and outer layer weights of a deep neural network," *IEEE Control Syst. Lett.*, vol. 6, pp. 1855–1860, 2022.
- [12] K. Esfandiari, F. Abdollahi, and H. Talebi, "A stable nonlinear in parameter neural network controller for a class of saturated nonlinear systems," *IFAC Proc. Vol.*, vol. 47, no. 3, pp. 2533–2538, 2014, 19th IFAC World Congress.
- [13] K. Esfandiari, F. Abdollahi, and H. A. Talebi, "Adaptive control of uncertain nonaffine nonlinear systems with input saturation using neural networks," *IEEE Trans. Neural Netw. and Learning Systems*, vol. 26, no. 10, pp. 2311–2322, 2015.
- [14] W. Gao and R. Selmic, "Neural network control of a class of nonlinear systems with actuator saturation," *IEEE Trans. Neural Netw.*, vol. 17, no. 1, pp. 147–156, 2006.
- [15] D. Rolnick and M. Tegmark, "The power of deeper networks for expressing natural functions," *arXiv:1705.05502*, 2018.
- [16] E. J. Griffis, O. S. Patil, Z. I. Bell, and W. E. Dixon, "Lyapunov-based long short-term memory (Lb-LSTM) neural network-based control," *IEEE Control Syst. Lett.*, vol. 7, pp. 2976–2981, 2023.
- [17] R. G. Hart, E. J. Griffis, O. S. Patil, and W. E. Dixon, "Lyapunov-based physics-informed long short-term memory (LSTM) neural network-based adaptive control," *IEEE Control Syst. Lett.*, vol. 8, pp. 13–18, 2024.
- [18] M. Ryu and K. Choi, "CNN-based end-to-end adaptive controller with stability guarantees," *arXiv*, 2024.
- [19] K. Esfandiari, F. Abdollahi, and H. A. Talebi, *Neural network-based adaptive control of uncertain nonlinear systems*, 2022nd ed. Cham, Switzerland: Springer Nature, Jun. 2021.
- [20] X. Zhou, H. Shen, Z. Wang, H. Ahn, and J. Wang, "Driver-centric lane-keeping assistance system design: A noncertainty-equivalent neuro-adaptive control approach," *IEEE/ASME Trans. Mechatron.*, vol. 28, no. 6, pp. 3017–3028, 2023.
- [21] M. Ryu, J. Kim, and K. Choi, "Imposing a weight norm constraint for neuro-adaptive control," *TechRxiv*, Oct. 2024, accepted for publication in 2025 23rd European Control Conference (ECC).
- [22] H. K. Khalil, *Nonlinear systems; 3rd ed.* Upper Saddle River, NJ: Prentice-Hall, 2002, the book can be consulted by contacting: PH-AID: Wallet, Lionel.
- [23] E. Arefinia, H. A. Talebi, and A. Doustmohammadi, "A robust adaptive model reference impedance control of a robotic manipulator with actuator saturation," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 50, no. 2, pp. 409–420, 2020.
- [24] W. He, Y. Dong, and C. Sun, "Adaptive neural impedance control of a robotic manipulator with input saturation," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 46, no. 3, pp. 334–344, 2016.
- [25] G. Peng, C. Yang, W. He, and C. L. P. Chen, "Force sensorless admittance control with neural learning for robots with actuator saturation," *IEEE Trans. Ind. Electron.*, vol. 67, no. 4, pp. 3138–3148, 2020.
- [26] S. Karason and A. Annaswamy, "Adaptive control in the presence of input constraints," *IEEE Trans. Autom. Control*, vol. 39, no. 11, pp. 2325–2330, 1994.
- [27] J. Nocedal and S. Wright, *Numerical optimization*, 2nd ed., ser. Springer series in operations research and financial engineering. New York, NY: Springer, 2006.
- [28] B. Evens, P. Latafat, A. Themelis, J. Suykens, and P. Patrinos, "Neural network training as an optimal control problem : — an augmented Lagrangian approach —," in *Proc. IEEE Conf. Decis. Control (CDC)*, 2021, 2021, pp. 5136–5143.
- [29] J. Wang, F. Yu, X. Chen, and L. Zhao, "ADMM for efficient deep learning with global convergence," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 111–119.
- [30] G. Taylor, R. Burmeister, Z. Xu, B. Singh, A. Patel, and T. Goldstein, "Training neural networks without gradients: A scalable admm approach," in *Int. Conf. Mach. Learn. (ICML)*, vol. 33, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 2016, pp. 2722–2731.
- [31] P. Kidger and T. Lyons, "Universal Approximation with Deep Narrow Networks," in *Conf. Learn. Theory (COLT)*, vol. 33, ser. Proceedings of Machine Learning Research, J. Abernethy and S. Agarwal, Eds., vol. 125. PMLR, 09–12 Jul 2020, pp. 2306–2327.
- [32] B. Sengupta, K. Friston, and W. Penny, "Efficient gradient computation for dynamical models," *Neuroimage*, vol. 98, pp. 521–527, 2014.
- [33] I. Douratsos and J. B. Gomm, "Neural network based model reference adaptive control for processes with time delay," *Int. J. Inf. Syst. Sci.*, vol. 3, no. 1, pp. 161–179, 2007.
- [34] M. Sauerens and A. Soquet, "Neural controller based on back-propagation algorithm," *IEEE Proc. Radar Signal Process.*, vol. 138, pp. 55–62, 1991.
- [35] C. A. Desoer and M. Vidyasagar, *Feedback Systems*. Society for Industrial and Applied Mathematics, 2009.
- [36] ROBOTIS, "OpenCR1.0," 2016, accessed: 2025-04-08.



Myeongseok Ryu received the B.S. degree in Mechanical Engineering from Incheon National University, South Korea, in 2023, and the M.S. degree in Mechanical Engineering from GIST, South Korea, in 2025. He is currently serving as a researcher at the Cho Chun Shik Graduate School of Mobility at KAIST, South Korea. His research interests include adaptive control, neural-networks, and constrained optimization.



Donghwa Hong received his B.S. degree in Physics and Engineering Physics from Yonsei University, Wonju, South Korea, in 2023. He is currently pursuing the M.S. degree in Mechanical Engineering from GIST, Gwangju, South Korea. His research interests include robot control, neural-networks, and model identification.



Kyunghwan Choi received his B.S., M.S., and Ph.D. degrees in Mechanical Engineering from KAIST, Daejeon, South Korea, in 2014, 2016, and 2020, respectively. Following his Ph.D., he worked at the Center for Eco-Friendly & Smart Vehicles at KAIST as a Postdoctoral Fellow and later as a Research Assistant Professor. In 2022, he joined GIST as an Assistant Professor in the Department of Mechanical and Robotics Engineering. Currently, he serves as an Assistant Professor at the Cho Chun Shik Graduate School of Mobility at KAIST, where

he is also the Director of the Mobility Intelligence and Control Laboratory. His research focuses on optimal and learning-based control for connected, automated, and electrified vehicles (CAEVs).